

Assignment #10 - Complete AI Dev Pipeline

No Safe Circle

Student & Game Overview

Student Name: Vincent J. Liguori

Capstone Game Title: No Safe Circle

Game Concept Brief: No Safe Circle is a fixed-isometric action dungeon game made in Unity. The player controls a wizard with the mouse, moves through dungeon spaces, fights enemies, uses health and mana, and interacts with doors and the environment. I used an AI-assisted development pipeline to break the game into smaller tasks, have AI agents work on those tasks, test the results, and keep track of what code and game systems came from each task.

Deliverable 1 - Playable Game

Playable Game Link: <https://cathode26.github.io/NoSafeCircle/index.html>

Status: The Unity Web build is published on GitHub Pages. It is still a rough prototype, but it can be opened in a browser without installing the Unity project.

Deliverable 2 - Pipeline Source Code & Unity Integration

Pipeline Repository: <https://github.com/cathode26/NoSafeCircle>

Pipeline Run Video: <https://cathode26.github.io/NoSafeCircle/video.html>

Target Game Engine: Unity 6 / Web (WebGL)

Automated Flow Description: The AI agents work directly on the Unity project instead of producing code or data that I later copy into the engine. They create or modify Unity C# scripts, tests, prefabs, scenes, and project files inside an isolated Git checkout. After the work is reviewed and verified, those exact Unity files are committed to the task branch and can run in the project without manual reformatting or copy/paste. My remaining human steps are design decisions, Unity runtime validation when needed, evidence review, merge approval, and Web publishing.

Main Parts of My AI Development Pipeline

I built the pipeline as several smaller systems instead of one giant AI agent. Each part has a specific job, and the AI models only get the access they need for that job.

Component	What It Does	Human Involvement
GDD	Defines how the game is supposed to work.	I write/approve the design.
GDD RAG	Finds the relevant parts of the GDD without always sending the whole document.	I decide when retrieval is appropriate.
TaskGraph	Stores game-development tasks, requirements, dependencies, ownership and evidence rules.	I approve important changes to the development plan.
Task Decomposition	Breaks large features into smaller AI-sized implementation tasks and has another model review the breakdown.	I step in for real design ambiguity and approve structure changes.
AgentRuntime / TaskExecution	Lets the pipeline use both Claude Code and OpenAI/Codex so the two providers can review each other, different jobs can use different strengths, and one provider's usage limits do not stop the whole workflow.	I configure the provider/model policies; the pipeline can route work between the two providers.
Software Architect	Polls the project state, checks dependencies and conflicts, recommends START / WAIT / HUMAN REVIEW, and recommends how the implementation work should be routed.	I start the supervised architect session and handle genuine design/GDD/task-authority questions.
GitHub Issue Workflow	Keeps task/branch/commit/handoff state outside any one AI chat session.	I post PASS/FAIL when runtime testing is required.
Game Task Agent	Runs one real task from checkout setup through an AI-generated, committed task branch and Unity handoff.	The Software Architect can launch it automatically; I step in when the task reaches a human-review or Unity runtime check.
ExecutionCrew	Uses separate AI roles for implementation, test writing, review and bounded repair.	Normally no manual coding step between those roles.
Unity Testing	Runs Edit Mode / Play Mode tests tied to an exact commit.	I run visual/interactive checks that tests cannot prove.
TaskDelivery	Packages proof that a task really shipped: files, commit, tests and human validation.	I review truth-bearing evidence claims.
Merge / Web Build	Integrates accepted work and publishes the playable build.	Merge and Web publishing are still human-controlled today.
Historical Context / Session Handoff	Keeps long-running project context in the repository so a new ChatGPT, Claude, or Codex session can continue without reconstructing the project from memory.	At the end of a substantial session I preserve/update the handoff; the next AI verifies live Git, TaskGraph, and GitHub state before continuing.
GDD Reconciliation / Design Refinement	Compares the GDD with the current Unity project and proposed work. Repeated runs expose missing requirements or ownership decisions, which can cause the GDD to be clarified and then checked again.	When the project cannot answer a real design question, I make the decision and update the GDD before the next reconciliation pass.

What Each Component Does

GDD - The Design Source

The Game Design Document is the starting point for the pipeline. It describes how No Safe Circle is supposed to work: gameplay rules, enemies, combat, rooms, doors, camera behavior, and the other systems in the game. I still treat the GDD as human-approved design authority. An AI can read it, search it, and propose work based on it, but an AI is not allowed to silently change the game design.

GDD RAG - Finding the Relevant Parts of the Design

I built a RAG system over the GDD so an AI does not always need the entire design document in its prompt. The RAG system creates a searchable index of the current GDD and can pull the sections most relevant to the task or review.

I tested this by having the same Claude model review the exact same task breakdown twice. One review received the full GDD and the other used the smaller RAG-based context. The RAG version used 39.5% fewer total tokens and cost 17.3% less, while both reviews returned PASS with no findings. I have tested this system successfully, although I have not made RAG mandatory for every decomposition run yet.

GDD Reconciliation / Design Refinement - Stress-Testing the Design

The GDD is not something I write once and then freeze. When I first turn the design into a TaskGraph, the reconciliation process compares the GDD with the current Unity project and tries to assign every required behavior to concrete development work.

That process often exposes decisions that were clear enough in a game-design document but not specific enough for several independent AI developers. For example, the pipeline exposed questions about who owns enemy health, player health, doorway-crossing state, active-enemy bookkeeping, navigation, restart state, and shared input behavior.

If the answer already exists in the GDD or Unity project, the proposed work can be corrected. If the design itself is genuinely ambiguous, the question comes back to me instead of letting an AI invent new game rules. I make the decision, update the canonical GDD, and run reconciliation/review again.

Because of this, creating the TaskGraph was highly iterative. I ran reconciliation and verification many times, and the GDD became more detailed as those passes turned previously implicit assumptions into explicit gameplay and ownership rules.

TaskGraph - The Development Plan

The TaskGraph is the main structure I built to describe the actual work needed to make the game. Instead of keeping a normal to-do list, each game task has an ID such as NSC-012 and contains the information needed to implement and prove that piece of the game.

- what feature needs to be implemented;
- its acceptance criteria;
- how it will be tested;
- which other tasks it depends on;
- which Unity files, scenes, prefabs, or systems it owns;
- which larger feature it belongs to; and
- what evidence is required before the task can be considered delivered.

The graph also records parent/child relationships and dependencies between tasks. The task describes what should be built, while Git commits, Unity tests, and delivery evidence are used later to prove that the feature was actually delivered.

Task Decomposition - Breaking Large Features Into AI-Sized Jobs

Some GDD features are too large to hand directly to one coding agent. I built a decomposition system that can take a large TaskGraph item and propose several smaller tasks that can each be implemented and tested separately.

The normal round-robin process can use both providers. OpenAI/Codex creates the first task breakdown and Claude reviews it. If Claude finds a real problem, Claude can revise the breakdown. Because Claude is now the author of that revision, it is not allowed to approve its own work, so Codex becomes the next independent reviewer. The providers can continue alternating until an independent reviewer passes the breakdown, the system reaches a real question for me, something fails, or the review limit is reached.

A reviewed decomposition is still a proposal. Important TaskGraph structure changes cross a human/authorization boundary before becoming real project work.

AgentRuntime / TaskExecution - One Interface for Claude and OpenAI

I wanted the pipeline to be able to use two competing AI providers instead of depending on only one. That lets Claude and OpenAI/Codex independently review each other's work, lets me choose the provider that is a better fit for a job, and keeps the whole pipeline from stopping if I run into one provider's usage or token limits.

AgentRuntime gives the rest of the pipeline one controlled way to call either Claude Code or OpenAI/Codex. Each request can specify the job the AI is doing, what project context it can see, what it is allowed to change, how much time or model effort it can use, and what kind of result the pipeline expects back.

The runtime also records the actual provider/model, token usage, failures, and raw provider output.

TaskExecution adds the No Safe Circle task identity on top of that generic runtime. This is what lets the rest of the pipeline switch between Claude and OpenAI without rewriting the whole workflow.

Software Architect - Choosing What Should Be Worked On

The Software Architect is the scheduling layer for the pipeline. I start one supervised architect session, and it repeatedly looks at the TaskGraph, dependencies, current branches and checkouts, other work already in progress, possible integration conflicts, and whether a task should be broken down further. Its job is not to write the game itself; its job is to decide what work is safe and useful to start next.

For each candidate task it can recommend START, WAIT, or HUMAN REVIEW. When it recommends START, it can also recommend how much AI reasoning the job needs and whether Claude, OpenAI, or no particular provider is preferred for the implementation. The architect is still advisory: the pipeline controls the actual task claim, branch, provider/model routing, repository changes, and worker launch. A real design, GDD, or task-definition question still comes back to me.

Historical Context / Session Handoff - Keeping AI Context Across Sessions

Because this project is too large to fit inside one AI conversation, I keep a repository-based session handoff so work can move between ChatGPT, Claude, and Codex without depending on chat memory.

CURRENT_CONTEXT.md gives a new session the current objective, important architecture rules, known issues, and next action, with focused historical handoffs available when more background is needed. Before making changes, the new agent must verify the current Git, TaskGraph, tests, and GitHub workflow state so older context can explain why decisions were made without overriding what is actually true in the project now.

GitHub Issue Workflow - Remembering Work Between AI Sessions

AI sessions end, so I did not want the current state of a task to live only inside one ChatGPT, Claude, or Codex conversation. The GitHub Issue workflow stores operational state such as task ownership, branch, commit, current phase, and the next human action.

If a task reaches a Unity runtime check, the Issue records the exact commit and checklist. I can post PASS or FAIL, and a later AI session can resume from that durable state instead of asking me to reconstruct what the previous agent was doing.

Game Task Agent - Running One Real Game Task

The Game Task Agent connects the planning side of the system to actual Unity development. Once a task is selected, it checks the TaskGraph and Git state, claims the task, creates or resumes a separate checkout and branch, inspects the Unity project, determines the smallest group of files that should change, validates that scope, runs the ExecutionCrew, verifies the result, applies the verified changes, commits and pushes the task branch, and then creates the Unity handoff when human testing is required.

The separate checkout matters because the AI can work on a feature without directly editing my main development branch.

ExecutionCrew - The AI Coding Team

ExecutionCrew is the part that actually changes the Unity project. I separated the work into different roles instead of asking one AI to write a feature and then grade its own work.

- Contract Locality Auditor - checks whether the requested job is actually small and self-contained enough to implement safely.
- Implementer - writes the Unity feature.
- Test Author - creates or updates the Unity tests for the task.
- Validator - reviews the resulting implementation without being the original implementer.

If the validator finds a specific problem, the pipeline can perform a limited repair cycle. The implementation provider can be Claude Code or OpenAI/Codex.

Patch Verification and Git Isolation

I do not let an AI statement such as 'I only changed these files' count as proof. The pipeline inspects the real Git changes and compares them with the file scope that was approved for the task.

The candidate can first be applied and checked in a disposable repository copy. Only the verified changes are then applied to the real task checkout. That separates what the AI claims it changed from what Git proves it changed.

Unity Automated Testing

The pipeline uses Unity's own test system. Depending on the feature, the task can have Edit Mode or Play Mode tests. The test runner preserves results and logs tied to the exact Git commit that was tested, so an old passing test cannot be reused as proof for newer code.

Human Unity Validation

This is one of the major places where I deliberately stay in the loop. Automated tests are useful, but they cannot reliably answer every game-development question.

For something visual or interactive, the pipeline gives me the exact commit, scene, Play Mode steps, and expected result. I open Unity and test the real game. If it works, I report PASS. If it fails, I report FAIL and describe what happened so the task can go back into a repair cycle. Camera feel, sprite sorting, door interaction, and other visual/gameplay behavior are examples of checks that can need this human step.

TaskDelivery and Evidence - Proving a Task Really Shipped

After implementation and Unity validation, TaskDelivery packages the proof for the task. The evidence can connect the TaskGraph requirement to the exact Git commit, Unity files, automated test XML/logs, and human Play Mode validation.

I still review the truth-bearing claims, such as which files actually prove a requirement and which test evidence belongs to each completion gate. Once the evidence is committed, the TaskGraph can derive the task's current conformance from that evidence instead of trusting an AI to simply say the task is done.

Merge and Web Build

The final game still lives in normal Git and Unity. After a task passes implementation, testing, human checks when required, and evidence closeout, it can be integrated back into the main game. Merge authority is still human-controlled in the current pipeline.

The Unity Web/WebGL build and GitHub Pages publishing step are also manual today. Those would be good future automation targets, but I intentionally did not start the project by giving an AI permission to merge and publish anything automatically.

How All of the Pieces Work Together

This is the current end-to-end shape of the pipeline, including where I still have to do something myself:

[HUMAN] Write and approve the GDD - I remain the design authority for the game.

[GDD RAG] Index and retrieve relevant design information - Used when a smaller, focused GDD context is appropriate.

[GDD RECONCILIATION] Compare the design with the current Unity project and proposed work - If a real design decision is missing, I update the GDD and run the analysis again. This loop may happen many times before the TaskGraph stabilizes.

[TASKGRAPH] Store features as structured tasks - Requirements, dependencies, Unity ownership, validation rules and evidence expectations live here.

[TASK DECOMPOSITION] Break down work that is too large - OpenAI/Codex can author the first breakdown.

[AI REVIEW] Independently review the task breakdown - Claude can review Codex; if Claude revises, Codex becomes the next reviewer.

[HUMAN] Approve/authorize important TaskGraph structure changes - AI proposals do not silently become the real project plan.

[SOFTWARE ARCHITECT] Recommend safe work to start and how to route it - Checks dependencies and current work, returns START, WAIT or HUMAN REVIEW, and can recommend Claude/OpenAI preference and reasoning level.

[PIPELINE] Choose the actual provider/model and claim the task - Deterministic code owns the real operation.

[GAME TASK AGENT] Create or resume an isolated Git checkout and task branch - The AI works away from main.

[AI SUPERVISOR + PIPELINE] Inspect the Unity project and approve the exact file scope - The task cannot casually expand into unrelated systems.

[EXECUTIONCREW - CLAUDE OR OPENAI] Implement, write tests, validate and repair - Different roles handle coding, tests and review.

[PIPELINE] Verify the real Git patch in isolation - The actual diff must match the approved task scope.

[GAME TASK AGENT] Apply the verified changes, commit and push the task branch - This creates the exact Unity version that will be tested.

[UNITY AUTOMATED TESTING] Run the required Edit Mode / Play Mode tests - Results are bound to the exact commit.

[HUMAN - WHEN REQUIRED] Open Unity and perform the runtime checklist - I report PASS or FAIL for visual/interactive behavior.

[AI + PIPELINE ON FAIL] Repair and validate again - A failed human/runtime check sends the task back for bounded repair.

[TASKDELIVERY ON PASS] Build delivery evidence - Connect the task to the exact Unity files, Git commit, tests and human result.

[HUMAN] Review evidence claims that require judgment - I approve what actually proves each requirement.

[TASKGRAPH] Derive the current conformance state from committed evidence - Completion comes from evidence, not from an AI self-report.

[HUMAN - CURRENTLY] Approve the final merge into the main game - Automatic merge authority is not enabled.

[HUMAN - CURRENTLY] Create the Unity Web/WebGL build and publish it - For this assignment I published the Web build to GitHub Pages.

Examples of Pipeline Work Already in the Game

Enemy Health and Defeat: The pipeline implemented enemy health, taking damage, defeat behavior, and reset behavior. The Unity Play Mode tests passed 15/15.

Encounter Enemy Limit: The pipeline added the encounter system that keeps the active enemy count from going over the allowed limit. The related Unity tests passed and also checked the active-enemy registry.

Isometric Sprite Sorting: The pipeline created the shared SpriteRenderer setup that lets the wizard, doors, walls, and other world objects sort correctly in the isometric view. The Unity tests passed 45/45, and I also checked the front/behind sorting in Play Mode.

Seamless Scalable Wall Tiling (NSC-042): The pipeline implemented scalable wall tiling for long isometric walls so the wall texture can extend cleanly without visible stretching or seams. This is the feature traced end-to-end in the assignment pipeline video.

Deliverable 3 - Pipeline Audit & Cost Analysis

1. What Did the Pipeline Produce?

The pipeline produced real Unity gameplay code and project content used in the playable project, including enemy health and defeat, encounter enemy limits, isometric sorting, and NSC-042 - Seamless Scalable Wall Tiling for Long Isometric Walls. The pipeline video traces NSC-042 from the task and AI implementation/test stages to the working Unity result.

2. What Manual Steps Are Still Left?

I still have to step in at a few points. I start the supervised Software Architect session, approve important design or TaskGraph structure decisions, run Unity when a feature needs a real Play Mode check, review the delivery evidence that requires human judgment, approve the final merge, and currently build and publish the Web version myself.

3. What Would It Take to Remove Those Manual Steps?

I would automate more of the remaining handoff work: run the authoritative Unity tests automatically, continue repair cycles after failed tests, finish evidence and merge closeout automatically when the decision is purely mechanical, and have a build job create and publish the Web version. I would still keep a person involved for design decisions and visual or gameplay checks that are hard to judge with automated tests.

4. Architectural Decision I Would Change

The architecture review found the core design sound but overbuilt. I built autonomous dispatch, execution routing, and parallel-worker tooling before proving enough real game tasks through the simpler delivery loop.

5. What I Would Do Instead

I would start with GDD -> TaskGraph -> ExecutionCrew -> Unity validation -> TaskDelivery -> human merge, and add scheduling or routing only after real development bottlenecks justified it.

6. Total Actual Run Cost

NSC-042 consumed 5,535,612 tokens across 16 recorded pipeline interactions. Historical artifacts preserve \$3.775176 in explicit provider cost (\$3.78 rounded) for 95.55% of the task's tokens. The remaining supervisor records preserve token counts but no dollar amount, so I report \$3.78 as the measured recorded cost of the demonstrated run rather than inventing charges for missing historical cost fields.

7. Most Expensive Pipeline Step

The most expensive step in the recorded NSC-042 run was the ExecutionCrew Implementer prompt using Claude Sonnet 5. It consumed 2,378,961 tokens and \$1.304855, about 43% of the task's total tokens and 34.6% of its recorded provider cost.

8. Is This Affordable for a Solo or Small Team?

Yes for direct feature work: NSC-042's measured recorded provider cost was \$3.78 even with separate contract auditing, implementation, test writing, and validation. Project-wide, my August reconstruction recovered at least 554.3 million tokens (513.6M Claude + 40.7M Codex), worth roughly \$180-\$325 at comparable provider token rates; subscription allowances mean that is an API-equivalent value, not necessarily the amount billed to me. I cannot reconstruct a trustworthy infrastructure-versus-game dollar split because not every provider history retained task attribution, but the architecture review showed that most effort was front-loaded into pipeline infrastructure while the playable game is only about 5% complete. The infrastructure is now largely finished, so future AI budget should go toward shipping game features rather than adding more orchestration.

Mid-Project Cost Reduction

One of the clearest improvements I made was reducing how much of the GDD I send to the AI reviewer. I ran a controlled comparison where the same task breakdown was reviewed twice by the same Claude model. One version received the full GDD, and the other received only the relevant GDD information through the project's RAG system.

	Before: Full GDD	After: GDD RAG	Change
Prompt size	243,821 bytes	197,562 bytes	19.0% smaller
Input tokens	392,429	233,319	40.5% fewer
Output tokens	10,730	10,490	2.2% fewer
Total tokens	403,159	243,809	39.5% fewer
API cost	\$0.6327	\$0.5235	17.3% cheaper
Result	PASS, no findings	PASS, no findings	Same result

Before: The reviewer always received the full GDD. **After:** The reviewer received the task information plus the most relevant GDD sections, and could still look up more information if it needed it.

Result: The shorter version used 39.5% fewer total tokens and cost 17.3% less, while producing the same PASS result with no findings on the same task breakdown.